

An Analysis of Overfitting machine learning

Overfitting machine learning

Overfitting machine learning -- Part 1

Underfitting is the inverse of overfitting, meaning that the statistical model or machine learning algorithm is too simplistic to accurately capture the patterns in the data. A sign of underfitting is that there is a high bias and low variance detected in the current model or algorithm used (the inverse of overfitting: low bias and high variance). This can be gathered from the Bias-variance tradeoff, which is the method of analyzing a model or algorithm for bias error, variance error, and irreducible error. With a high bias and low variance, the result of the model is that it will inaccurately represent the data points and thus insufficiently be able to predict future data results (see Generalization error). As shown in Figure 5, the linear line could not represent all the given data points due to the line not resembling the curvature of the points. One would expect to see a parabola-shaped line as shown in Figure 6 and Figure 1. If Figure 5 were to be used for analysis, false predictive results would be given contrary to the results if Figure 6 was analyzed. Burnham & Anderson state the following.

Overfitting machine learning -- Part 2

Usually, a learning algorithm is trained using some set of "training data": exemplary situations for which the desired output is known. The goal is that the algorithm will also perform well on predicting the output when fed "validation data" that was not encountered during its training. Overfitting is the use of models or procedures that violate Occam's razor, for example by including more adjustable parameters than are ultimately optimal, or by using a more complicated approach than is ultimately optimal. For an example where there are too many adjustable parameters, consider a dataset where training data for y can be adequately predicted by a linear function of two independent variables. Such a function requires only three parameters (the intercept and two slopes). Replacing this simple function with a new, more complex quadratic function, or with a new, more complex linear function on more than two independent variables, carries a risk: Occam's razor implies that any given complex function is a priori less probable than any given simple function. If the new, more complicated function is selected instead of the simple function, and if there was not a large enough gain in training data fit to offset the complexity increase, then the new complex function "overfits" the data and the complex overfitted function will likely perform worse than the simpler function on validation data outside the training dataset, even though the complex function performed as well, or perhaps even better, on the training dataset. When comparing different types of models, complexity cannot be measured solely by counting how many parameters exist in each model; the

expressivity of each parameter must be considered as well. For example, it is nontrivial to directly compare the complexity of a neural net (which can track curvilinear relationships) with m parameters to a regression model with n parameters. Overfitting is especially likely in cases where learning was performed too long or where training examples are rare, causing the learner to adjust to very specific random features of the training data that have no causal relation to the target function. In this process of overfitting, the performance on the training examples still increases while the performance on unseen data becomes worse. As a simple example, consider a database of retail purchases that includes the item bought, the purchaser, and the date and time of purchase. It's easy to construct a model that will fit the training set perfectly by using the date and time of purchase to predict the other attributes, but this model will not generalize at all to new data because those past times will never occur again. Generally, a learning algorithm is said to overfit relative to a simpler one if it is more accurate in fitting known data (hindsight) but less accurate in predicting new data (foresight). One can intuitively understand overfitting from the fact that information from all past experience can be divided into two groups: information that is relevant for the future, and irrelevant information ("noise"). Everything else being equal, the more difficult a criterion is to predict (i.e., the higher its uncertainty), the more noise exists in past information that needs to be ignored. The problem is determining which part to ignore. A learning algorithm that can reduce the risk of fitting noise is called "robust."

Overfitting machine learning -- Part 3

A function that is overfitted is likely to request more information about each item in the validation dataset than does the optimal function; gathering this additional unneeded data can be expensive or error-prone, especially if each individual piece of information must be gathered by human observation and manual data entry. A more complex, overfitted function is likely to be less portable than a simple one. At one extreme, a one-variable linear regression is so portable that, if necessary, it could even be done by hand. At the other extreme are models that can be reproduced only by exactly duplicating the original modeler's entire setup, making reuse or scientific reproduction difficult. It may be possible to reconstruct details of individual training instances from an overfitted machine learning model's training set. This may be undesirable if, for example, the training data includes sensitive personally identifiable information (PII). This phenomenon also presents problems in the area of artificial intelligence and copyright, with the developers of some generative deep learning models such as Stable Diffusion and GitHub Copilot being sued for copyright infringement because these models have been found to be capable of reproducing

An Analysis of Overfitting machine learning

Overfitting machine learning

certain copyrighted items from their training data.

Overfitting machine learning -- Part 4

Increase the complexity of the model: If the model is too simple, it may be necessary to increase its complexity by adding more features, increasing the number of parameters, or using a more flexible model. However, this should be done carefully to avoid overfitting. Use a different algorithm: If the current algorithm is not able to capture the patterns in the data, it may be necessary to try a different one. For example, a neural network may be more effective than a linear regression model for some types of data. Increase the amount of training data: If the model is underfitting due to a lack of data, increasing the amount of training data may help. This will allow the model to better capture the underlying patterns in the data. Regularization: Regularization is a technique used to prevent overfitting by adding a penalty term to the loss function that discourages large parameter values. It can also be used to prevent underfitting by controlling the complexity of the model. Ensemble Methods: Ensemble methods combine multiple models to create a more accurate prediction. This can help reduce underfitting by allowing multiple models to work together to capture the underlying patterns in the data. Feature engineering: Feature engineering involves creating new model features from the existing ones that may be more relevant to the problem at hand. This can help improve the accuracy of the model and prevent underfitting.